

[Listing 10-10](#) shows how we can mark our `Shape` class as an abstract base class and indicate its abstract methods.

File: InheritedShapes/Shape.py

```
# Shape class
#
# To be used as a base class for other classes

import random
from abc import ABC, abstractmethod

# Set up the colors
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)

❶ class Shape(ABC): # identifies this as an abstract base class

    ❷ def __init__(self, window, shapeType, maxWidth, maxHeight):
        self.window = window
        self.shapeType = shapeType
        self.color = random.choice((RED, GREEN, BLUE))
        self.x = random.randrange(1, maxWidth - 100)
        self.y = random.randrange(25, maxHeight - 100)

    ❸ def getType(self):
        return self.shapeType

    ❹ @abstractmethod
        def clickedInside(self, mousePoint):
            raise NotImplementedError

    ❺ @abstractmethod
        def getArea(self):
            raise NotImplementedError

    ❻ @abstractmethod
        def draw(self):
            raise NotImplementedError
```

Listing 10-10: The `Shape` base class that inherits from `ABC` with abstract methods